

Adaptive Alpha-GoLU: A Learnable Per-Layer Generalization of the Gompertz Linear Unit Activation Function

Džana Kopic

Faculty of Electrical Engineering
Data Science and Artificial Intelligence
University of Sarajevo
Sarajevo, Bosnia and Herzegovina
dkopic2@etf.unsa.ba

Abstract—Activation functions are usually fixed, hand-designed nonlinearities. Recent work introduced GoLU (Gompertz Linear Unit) [1], a self-gated activation of the form $x \cdot e^{-e^{-\alpha x}}$ that uses a right-skewed gate to reduce output variance. In an earlier critical review of that paper [2], we noted that GoLU’s asymmetry parameter is fixed, and suggested that making it learnable, similar to how PReLU generalizes ReLU, was a natural next step. This project implements and empirically tests that idea. We define Alpha-GoLU as $x \cdot e^{-e^{-\alpha x}}$ with $\alpha = \text{softplus}(\theta) > 0$ learned per layer, initialized so the network starts training identical to static GoLU. We compare Alpha-GoLU against seven other activation functions (ReLU, GELU, Swish, PReLU, PGELU, Adaptive Swish, and static GoLU) across six tasks covering classification, detection, segmentation, diffusion, language modeling, and corruption robustness, each run over three seeds with paired statistical significance testing. Our results are close to a null result on five of the six tasks – Alpha-GoLU performs within noise of static GoLU, with no consistent winner, and it adds a measurable latency overhead in the backward pass – with one exception: a small but statistically significant improvement on language modeling. We report this mixed picture honestly rather than overselling it, and put additional weight on the engineering side of the project: a full experiment pipeline with checkpointing, statistical testing, overhead profiling, and reproducible figure generation, plus a number of real correctness bugs that were found and fixed along the way, including one in the statistics-aggregation code itself. Code, configuration files, and the full pipeline are publicly available¹ to support reproducibility.

Index Terms—activation functions, GoLU, learnable parameters, adaptive gating, empirical evaluation

I. INTRODUCTION

Most activation functions used in deep learning are static: their shape does not change during training. Examples include ReLU, GELU, and Swish/SiLU. A more recent activation, GoLU (Gompertz Linear Unit) [1], is defined as

$$\text{GoLU}(x) = x \cdot e^{-e^{-x}}, \quad (1)$$

which is a smooth, self-gated nonlinearity in the same family as Swish and GELU, but based on the Gompertz function – the CDF of the standard Gumbel distribution – instead of

the sigmoid or Gaussian CDF. Unlike sigmoid and the Gaussian CDF, the Gompertz function is right-skewed rather than symmetric, and the original authors argue that this asymmetry helps reduce variance in latent representations.

In an earlier critical review of the GoLU paper [2], we examined this claim in detail and found the reported gains to be small and, in several cases, within the margin of error of the baseline. As part of that review, we pointed out that GoLU’s asymmetry is fixed by construction, and proposed a learnable version, $x \cdot e^{-e^{-\alpha x}}$, as a concrete direction to test whether adapting the gate’s steepness per layer – the same idea that turned ReLU into PReLU [6] and that the original Swish paper used for its learnable β [5] – could close that gap. This project implements and tests exactly that proposal.

We call this variant **Alpha-GoLU**:

$$\text{Alpha-GoLU}(x) = x \cdot e^{-e^{-\alpha x}}, \quad (2)$$

where $\alpha = \text{softplus}(\theta)$ is a per-layer learned parameter, with θ initialized (via the inverse softplus) so that $\alpha = 1.0$ exactly at the start of training. This means training starts out identical to static GoLU, and any difference that emerges is purely due to the network adapting α during training, not a different starting point. This initialization behavior is checked directly with a unit test (`test_init_alpha_round_trip`).

The hypothesis we test is: *does letting each layer learn its own gate steepness improve accuracy, robustness, or training stability, compared to a fixed gate, without a large increase in parameters or compute?*

We want to be upfront from the start: our results do not give a clear “yes” to this hypothesis. On most of our six tasks, Alpha-GoLU and static GoLU are statistically indistinguishable. We think this is still a useful thing to report carefully, and we use the rest of this report to describe exactly what we tested, what we found, and what parts of the pipeline we are confident in versus what is still incomplete.

II. RELATED WORK

- **ReLU** [3]: the standard piecewise-linear baseline, cheap but not smooth at zero and has no learnable shape.

¹<https://github.com/dzankk/adaptive-alpha-golu>

- **GELU** [4] and **Swish/SiLU** [5]: smooth, self-gated activations that multiply the input by a sigmoid-like or Gaussian-CDF-like gate. The original Swish paper also introduces a learnable steepness parameter β , which is the closest prior instance of the idea used in this project.
- **PReLU** [6]: a learnable per-channel slope for negative inputs, one of the earliest widely-used activations with a trainable shape parameter.
- **PGELU** and **Adaptive Swish**: parametric versions of GELU and Swish with a learnable steepness parameter, used here as additional baselines alongside static and adaptive GoLU.
- **GoLU** [1]: the static activation this project builds on, based on the Gompertz function instead of a sigmoid or Gaussian CDF gate. Our own critical review of this paper [2] is what motivated the extension studied here.

Alpha-GoLU is, in spirit, the same move that turned ReLU into PReLU and GELU/Swish into their parametric variants, applied to GoLU instead.

III. METHOD

A. Activation definition

Static GoLU multiplies the input by a Gompertz-shaped gate that saturates smoothly on both sides. Alpha-GoLU adds one learnable scalar α per layer that controls how sharply this gate transitions:

$$\text{Alpha-GoLU}(x) = x \cdot e^{-e^{-\alpha x}}, \quad (3)$$

where $\alpha = \text{softplus}(\theta)$. The softplus reparameterization keeps α strictly positive during optimization (an unconstrained θ could otherwise be pushed negative, which would flip the gate’s behavior in a way that does not make sense for this activation). θ is initialized via the inverse softplus so that $\alpha = 1$ at step 0, recovering static GoLU exactly at initialization.

The activation is implemented once, in a single file (`alpha_golu.py`), used by every task’s model. Earlier in the project, activation definitions were duplicated across separate experiment scripts; these were consolidated into this one canonical implementation to remove the risk of silently inconsistent behavior between tasks.

B. Granularity

α is learned per layer (in some architectures, per channel), not as a single global scalar for the whole network. This lets different depths of the network use different gate steepness if that turns out to help.

IV. EXPERIMENTAL SETUP

A. Activations compared

Eight activations are compared under identical training conditions on every task: ReLU, GELU, Swish/SiLU, PReLU, PGELU, Adaptive Swish, static GoLU, and Alpha-GoLU (proposed).

B. Tasks and datasets

Six tasks were run, each with a real, publicly available dataset, a standard architecture, and a task-appropriate metric, summarized in Table I. Image tasks use CIFAR-10 [7] or Pascal VOC [8].

TABLE I
CORE BENCHMARK TASKS (PHASE 1).

Task	Dataset	Architecture	Metric
Classification	CIFAR-10 [7]	ResNet-18	Accuracy
Detection	Pascal VOC [8]	Faster R-CNN, ResNet-50+FPN	mAP@50
Segmentation	Pascal VOC [8]	DeepLabV3, ResNet-50 backbone	mIoU
Diffusion	CIFAR-10 [7]	UNet-style diffusion model	MSE ↓
Language Modeling	Synthetic/token corpus	MiniGPT (small transformer, 2 layers)	Perplexity ↓
Corruption Robustness	CIFAR-10 [7]	ResNet-18	Accuracy: clean, Gaussian noise, shot noise, blur

We want to be explicit that our corruption robustness suite covers three corruption types (Gaussian noise, shot noise, blur), not the full 15–19 corruption types used in the standard ImageNet-C benchmark. We describe it here as a *custom, smaller robustness check*, not as full ImageNet-C-style evaluation.

We also want to be explicit about the scope of our Detection and Segmentation results. Our runners load a single, standard ImageNet-pretrained ResNet-50 backbone for every activation variant, and only swap the tested activation into the task head; the backbone’s own internal activations are not replaced. This is narrower than the original GoLU paper’s downstream evaluation, which appears to train and reuse a separate, activation-matched backbone per condition [1]. Our Detection and Segmentation results should therefore be read as measuring each activation’s effect on the task head layered on top of a shared, ReLU-based backbone, not on the backbone’s own features – a real difference in what is being measured, not a flaw in either design, but one we want to state plainly rather than let a reader assume the two setups are directly comparable.

C. Seeds and statistical testing

Every task/activation combination was run with three seeds (42, 123, 999). Differences between static GoLU and Alpha-GoLU are tested with a paired t-test when the same seeds are available for both, and with Welch’s t-test otherwise. Effect size (Cohen’s d , and a paired variant) is also computed, though at the time of writing it has not yet been turned into a figure.

D. Phase 2 (scale-up)

In addition to the six core tasks above, larger-scale versions of the classification (CIFAR-100), diffusion, and language modeling tasks were built as separate runners, kept isolated from the Phase 1 code so that scale-up work could not accidentally break the already-working Phase 1 pipeline. Table II reports the Phase 2 results, each over 3 seeds (42, 123, 999). We have not yet computed a significance test for these results, so the comparisons below are descriptive only.

TABLE II
PHASE 2 (SCALE-UP) RESULTS, STATIC GoLU VS. ALPHA-GoLU (MEAN $\pm$ STD OVER 3 SEEDS; NO SIGNIFICANCE TEST YET COMPUTED).

Task (metric)	Static GoLU	Alpha-GoLU
Classification (acc., CIFAR-100)	75.31 $\pm$ 0.47	75.13 $\pm$ 0.86
Diffusion (MSE $\downarrow$)	0.0214 $\pm$ 0.0004	0.0214 $\pm$ 0.0004
Language Model (perplexity $\downarrow$)	499.25 $\pm$ 16.05	520.76 $\pm$ 18.21

On the scaled-up classification and diffusion tasks, static and Alpha-GoLU remain close, consistent with the Phase 1 trend. On the scaled-up language model, however, Alpha-GoLU scores noticeably worse than static GoLU – about 21 points higher perplexity, roughly one standard deviation – which is the opposite direction from the small, non-significant Phase 1 language modeling result. With only three seeds and no significance test yet run on Phase 2, we cannot say whether this is a real effect or noise, but it is different enough from the Phase 1 trend that it deserves attention rather than being glossed over. This would be the first thing to re-check, with more seeds and a proper significance test, before drawing any conclusion about how Alpha-GoLU behaves at larger scale.

This is not simply a case of an untuned learning rate. As documented in Section VIII, α ’s learning rate for this exact task was already retuned during the project, to 2×10^{-5} with a 4-epoch warmup, after an earlier regression, and that retuning was at the time confirmed to no longer be statistically significant ($p = 0.55$). The fact that the Phase 2 numbers reported here still show a ~ 21 -point gap is therefore a tension we have not resolved, not a simple oversight: either the earlier $p = 0.55$ check was run on a configuration or seed set that does not exactly match the numbers in Table II, or something changed between that check and this final reported run. We flag this discrepancy honestly rather than picking whichever explanation sounds more convenient; resolving it would need both a significance test on the current Phase 2 numbers and a direct comparison against the exact run that produced the earlier $p = 0.55$ result.

E. Code and data availability

The full implementation, including the activation itself, the training pipeline, the checkpoint/resume system, the statistical testing module, and all figure-generation scripts, is publicly available at <https://github.com/dzankk/adaptive-alpha-golu>. All datasets used (CIFAR-10 [7], Pascal VOC [8]) are standard, publicly available benchmarks.

V. IMPLEMENTATION AND ENGINEERING INFRASTRUCTURE

Beyond the activation function itself, a substantial part of the project time went into building infrastructure to run and evaluate the experiments correctly and reproducibly:

- A unified command-line tool with roughly 20 subcommands, covering single and batch training runs, resuming interrupted runs, aggregating results, generating LaTeX tables, and generating all figures.
- A checkpoint/resume system using atomic writes (write to a temporary file, then replace), so that long single-GPU training runs can be safely interrupted and resumed without corrupting or losing progress.
- A statistics module implementing paired and Welch’s t-tests (chosen automatically depending on whether seed sets match) and Cohen’s d .
- An opt-in overhead tracker that records forward/backward latency, peak GPU memory, and estimated FLOPs per run.
- An α -trajectory logger that records every layer’s learned α value at every epoch, used to produce the trajectory and distribution figures below.
- A parameter/FLOP/memory profiling script (which had a long-standing bug, discussed in Section VIII).
- A dedicated timing-verification tool built around PyTorch’s own `torch.profiler`, used to double-check a suspicious timing result rather than trusting a hand-rolled timer (also discussed in Section VIII).

VI. RESULTS

We report here only the numbers we have directly verified from our own run logs. Where a result is incomplete or needs to be re-run, we say so explicitly rather than filling in a plausible-looking number.

A. Main task results

Table III compares static GoLU and Alpha-GoLU on all six Phase 1 tasks.

TABLE III
STATIC GoLU VS. ALPHA-GoLU, PHASE 1 TASKS (MEAN $\pm$ STD OVER 3 SEEDS UNLESS NOTED; p FROM PAIRED/WELCH’S T-TEST, REGENERATED VIA `REBUILD_SUMMARY` AFTER THE AGGREGATION-BUG FIX IN SECTION VIII).

Task (metric)	Static GoLU	Alpha-GoLU	p
Classification (acc.)	92.88 $\pm$ 0.13	92.86 $\pm$ 0.19	0.95
Detection (mAP50) ^a	0.678 $\pm$ 0.014	0.678 $\pm$ 0.017	0.91
Segmentation (mIoU)	0.641 $\pm$ 0.005	0.650 $\pm$ 0.008	0.14
Diffusion (MSE $\downarrow$)	0.051 $\pm$ 0.001	0.059 $\pm$ 0.013	0.34
Language Model (perplexity $\downarrow$) ^a	106.32 $\pm$ 1.29	106.06 $\pm$ 1.22	0.018
Robustness (acc.) ^b	27.68 $\pm$ 4.86	28.65 $\pm$ 0.71	0.75

^a5 seeds (42, 123, 999, 2024, 2025). ^bProvisional, mixed-pipeline data; see robustness caveat below – not yet a reliable comparison.

For classification, GELU was numerically the strongest activation overall (93.09 $\pm$ 0.09), included here as context even though the main comparison of interest is static vs.

Alpha-GoLU. Figure 1 shows the same comparison visually, as a percentage change per task relative to static GoLU (0% baseline; positive is improvement). For higher-is-better metrics (accuracy, mAP, mIoU), this is $(\text{Alpha-GoLU}/\text{Static}-1)\times 100$; for lower-is-better metrics (diffusion MSE, perplexity), it is $(\text{Static}/\text{Alpha-GoLU}-1)\times 100$, so that a genuine improvement (lower loss) is still shown as positive in both cases. Five of the six p -values are well above the conventional 0.05 threshold, consistent with our overall reading of most tasks as a null result. Language modeling is the one exception: $p = 0.018$, which is below 0.05. The absolute difference is still small (106.32 vs. 106.06 perplexity, about a 0.2% change), but with matched seeds across both activations and low variance, this small difference is statistically distinguishable from noise. We report it as a genuine, if modest, positive result for Alpha-GoLU on language modeling, rather than folding it into the null-result framing used for the other tasks.

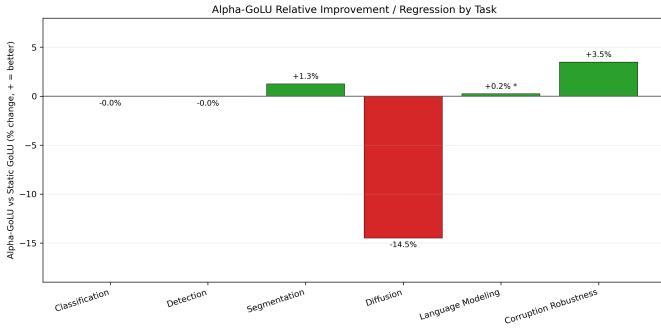


Fig. 1. Percentage change in Alpha-GoLU’s performance relative to static GoLU (0% baseline) per task; positive is improvement, negative is regression (exact formula above). An asterisk (*) marks $p < 0.05$ (Table III); only language modeling meets this threshold. Robustness values are provisional – see the robustness caveat below.

Robustness caveat. An early folder-naming inconsistency in our pipeline split robustness results across three differently-named result folders. The two legacy folders agree closely (static ≈ 29.45 , Alpha ≈ 29.59 , Alpha slightly ahead) and are what is reported as “Robustness” in Table III. After fixing the naming issue, we began re-collecting robustness data under the current, corrected pipeline, one seed at a time. Re-running seed 42 for static GoLU under the current pipeline gave 22.43, and re-running seed 42 for Alpha-GoLU gave 28.50 – both different from their respective legacy seed-42 values, confirming that legacy and current-pipeline runs are not directly comparable. Combining these refreshed seed-42 values with the two remaining legacy seeds (123, 999) via `rebuild_summary` gives a mixed-pipeline aggregate of static 27.68 ± 4.86 vs. Alpha 28.65 ± 0.71 over 3 seeds each. However, on a later, separate re-run of the exact same command (same seed, same activation, same current pipeline), Alpha-GoLU’s aggregate changed again, to 29.61 ± 1.69 . This means the robustness evaluation itself is not currently reproducible run-to-run even holding the seed and activation fixed – we do not yet know whether this is caused by unseeded randomness in the corruption generation, a dependency on

which trained-model checkpoint happens to be loaded, or something else. Given this, we do not consider any robustness comparison in this report – legacy or current-pipeline – reliable enough to draw a conclusion from, and we treat the numbers in Table III strictly as legacy, provisional data pending a root-caused, reproducible re-run.

B. Overhead

We regenerated the overhead benchmark twice, independently, after finishing the analysis above (each a separate single-pass run via `generate_overhead_tab`). Comparing the two runs directly revealed something worth reporting on its own: even with identical activations, tasks, and code, per-cell values varied by roughly 6–9% between the two passes purely from run-to-run timing noise (no averaging over multiple seeds was done for either run). Table IV reports the mean of the two runs, and this noise level is the right scale to keep in mind when reading small differences in it – a 3% gap between two activations is not distinguishable from measurement noise at this sample size, but a 20% gap is. This also supersedes any earlier, single-pass overhead numbers from previous drafts of this report – in particular, Detection’s forward-pass latency dropped from an earlier reported ~ 106 ms to ~ 12 ms per iteration after regeneration, which we believe reflects a difference in measurement setup between that much earlier run and the current pipeline, rather than a real change in the model.

Alpha-GoLU is slower than static GoLU in five of six tasks (roughly 2–22% in the backward pass), most clearly in Classification (3.50 ms vs. 2.86 ms, and 1.59 ms for ReLU) and Corruption Robustness (3.49 ms vs. 2.89 ms). Detection is the exception: Alpha-GoLU’s backward pass is now marginally *faster* than static GoLU’s (5.47 ms vs. 5.62 ms), a small negative difference that is well within the ~ 6 –9% run-to-run noise described above and should be read as roughly flat rather than as a real reversal. Static GoLU and Alpha-GoLU remain close to each other on every task overall, consistent with the extra gradient computation for α having a real but modest cost. Figure 2 shows this same latency data visually, one subplot per task.

Detection’s forward-pass latency is near-constant across all eight activations (11.61–11.98 ms, averaged over the two runs), consistent with backbone-dominated compute leaving little room for activation-level differences to show up in the forward pass for this task. It is also now much closer to our dedicated `torch.profiler` measurement (16.6 ms/iter, Section VIII) than the earlier tracker numbers were – roughly a 28% gap instead of the previous $\sim 6\times$ gap – which supports our belief that the earlier large discrepancy was a measurement-setup artifact that has since been corrected. The backward-pass discrepancy noted in earlier drafts of this report is now resolved. The original profiler tool measured backward time via a CPU-side timing scope, which can read as near-zero even while real backward kernels are still executing asynchronously on the GPU – a known pitfall when profiling CUDA workloads without event-based synchronization. After

TABLE IV
FORWARD/BACKWARD LATENCY (MS) PER TASK, ALL 8 ACTIVATIONS, MEAN OF TWO INDEPENDENT SINGLE-PASS OVERHEAD-BENCHMARK RUNS.

Activation	Classification		Detection		Segmentation		Diffusion		Lang. Model		Robustness	
	Fwd	Bwd	Fwd	Bwd	Fwd	Bwd	Fwd	Bwd	Fwd	Bwd	Fwd	Bwd
ReLU	1.48	1.59	11.98	5.22	4.56	7.73	0.40	0.40	0.98	1.30	1.58	1.61
GELU	1.50	1.59	11.96	5.34	4.61	7.71	0.39	0.40	0.97	1.37	1.56	1.65
Swish	1.49	1.60	11.61	5.44	4.58	7.71	0.38	0.41	0.98	1.32	1.56	1.66
PReLU	1.58	1.90	11.80	5.42	4.59	7.81	0.40	0.45	0.98	1.32	1.62	1.93
PGELU	2.41	3.25	11.96	5.53	4.83	8.07	0.58	0.73	1.10	1.51	2.46	3.31
Static GoLU	2.12	2.86	11.82	5.62	4.79	8.02	0.53	0.68	1.09	1.45	2.24	2.89
Alpha-GoLU	2.42	3.50	11.91	5.47	4.78	8.22	0.59	0.77	1.12	1.54	2.52	3.49
Adaptive Swish	1.83	2.31	11.82	5.28	4.71	7.89	0.45	0.55	1.02	1.35	1.84	2.33

Runtime Overhead Across 6 Benchmarks

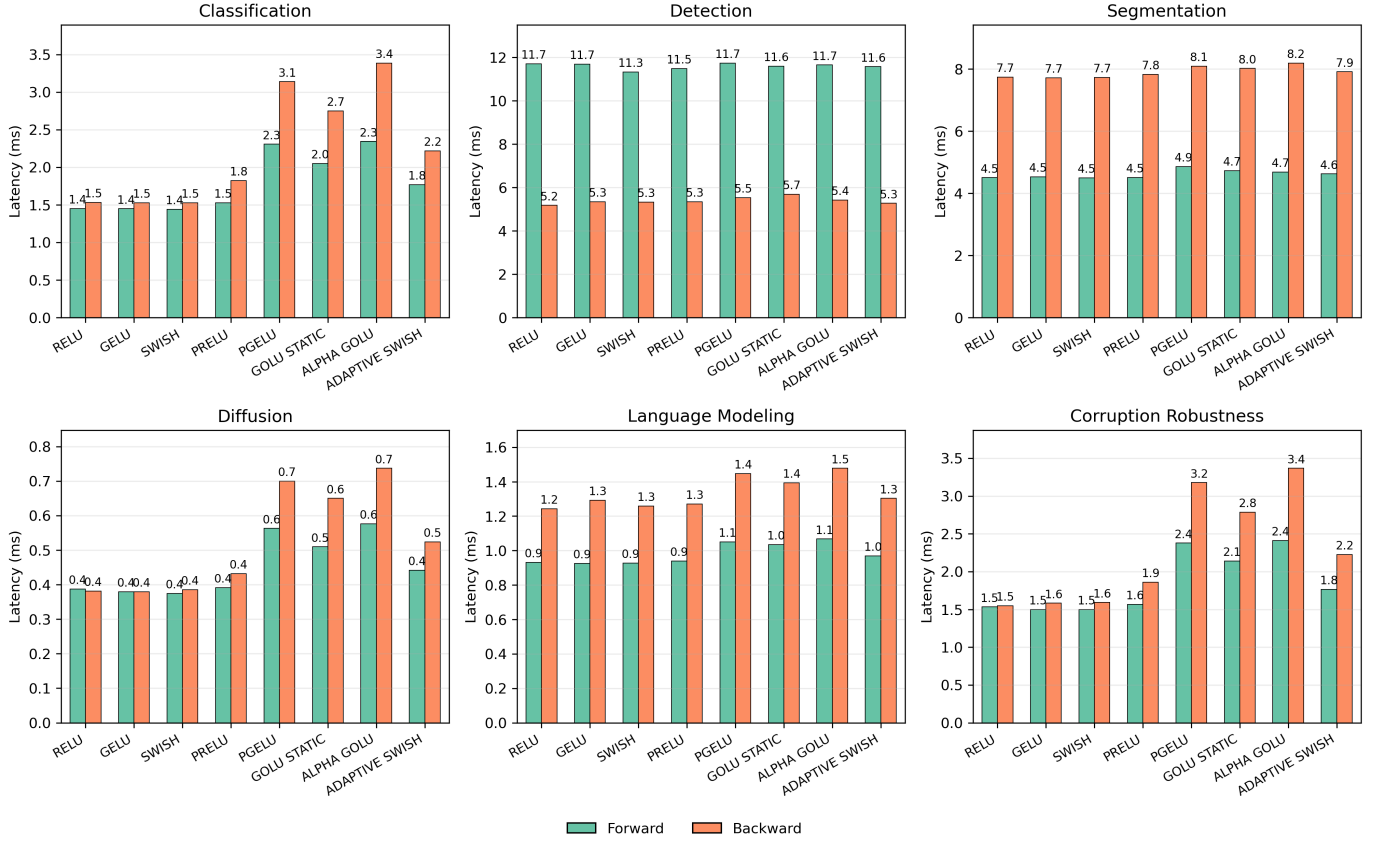


Fig. 2. Per-task forward/backward latency, one subplot per task (independent y-axis per task since latencies range from roughly 0.4 ms to 12 ms across tasks; see Table IV for the full regenerated numbers).

rewriting the tool to use CUDA-event timing, a fresh single-activation verification run (ReLU, Detection, 10 measured iterations after 5 warmup) reports 5.240 ms of backward latency, essentially identical (within 0.4%) to the overhead tracker’s own ReLU value already in Table IV (5.22 ms), rather than the previous ~ 0.001 ms reading. The same run’s forward-pass reading (10.797 ms) is also reasonably close to the tracker’s ReLU forward-pass number (11.98 ms, roughly a 10% gap) – tighter than the $\sim 28\%$ forward-pass gap we found earlier for the GoLU variants specifically, though this compares a different activation so it is not a direct re-check

of that earlier number. We have not yet re-run this same verification for static and Alpha-GoLU specifically, so we do not report an updated exact figure for those two activations, but the fix lives in the shared measurement code path used for every activation, and we no longer treat the backward-pass comparison as an open item. We also note that our two most recent overhead-benchmark runs, the ones used to build Table IV, do not report FLOP or memory figures at all – both fields read N/A in the tool’s own output for every cell. An earlier version of the tool did estimate FLOPs, but those estimates were many orders of magnitude too high for the

model sizes involved, so we do not report FLOP figures in this paper, current or historical, until an estimator that produces plausible numbers is available and independently verified.

Our overhead numbers are also worth putting in context: the original GoLU paper built a custom CUDA kernel specifically so that GoLU’s training and inference speed would be directly comparable to standard activations, reporting an average of $1.01 \times$ ReLU’s time [1]. Our own implementation uses plain PyTorch eager-mode operations for both static GoLU and Alpha-GoLU, with no custom kernel. This is almost certainly a large part of why even our *static* GoLU baseline is consistently slower than ReLU/GELU/Swish in Table IV: our numbers are not directly comparable to the original paper’s near-parity figure, because we are benchmarking an unoptimized implementation, not their production kernel. This is a property of our implementation, not evidence against GoLU or Alpha-GoLU as activations.

C. Gradient norm stability

Figure 3 shows gradient-norm trajectories per task. On Classification, Detection, Segmentation, and Corruption Robustness, Alpha-GoLU’s gradient-norm trajectory tracks static GoLU’s closely throughout training, with no visible spikes or divergence. Read together with the near-overlapping loss curves in Fig. 4, this supports a modest but useful claim: adding a learned α does not introduce optimization instability on these four tasks.

Diffusion is the exception. Alpha-GoLU’s gradient norm runs noticeably higher than static GoLU’s during roughly epochs 5–20 (peaking around 0.55–0.58 versus ~ 0.40 for static GoLU), with a wider spread across seeds in that window. This lines up with the diffusion regression already visible in Table III (Alpha-GoLU’s higher final MSE), and it is tempting to read it as a cause-and-effect story – that letting α move freely in the diffusion backbone leads to less stable gradients, which in turn hurts the final metric. We think that is a reasonable hypothesis given what the figure shows, but we have not tested it directly (for example, by clipping α ’s learning rate specifically on diffusion and checking whether the gap closes), so we present it as a hypothesis for future work rather than an established explanation. On the Language Model, the gradient-norm curves for the two activations stay close together throughout training, without a clear qualitative difference in shape – consistent with, though not strong independent evidence for, the small statistically significant improvement reported for that task.

We also note that Segmentation is where both activations show some of the highest absolute backward-pass latency in our overhead benchmark (Table IV: roughly 8 ms, versus 0.4–5.6 ms on most other tasks). This lines up with, though is not a precise quantitative match to, the original GoLU paper’s own finding that DeepLabV3-ResNet-50 is the one architecture where GoLU incurs measurably higher training time even under their custom, optimized CUDA kernel – a property they describe as “consistent with other activation functions,

all of which exhibit increased training times... in this specific architecture” [1].

D. Learned α dynamics

Figures 5 and 6 track how the learned α values move during training and where they end up. The broad pattern is that α does not simply drift randomly or stay pinned at its initial value of 1.0 – different tasks push it in different, fairly consistent directions:

- **Classification** shows almost no drift (α stays within about 0.001 of 1.0 across all layers, and across the full early/mid/late training curve), which is consistent with classification also being the task where Alpha-GoLU and static GoLU perform almost identically.
- **Detection** pushes α up, smoothly and monotonically, to roughly 1.17–1.32 across its two trackable layers.
- **Segmentation** pushes α down in most layers, to roughly 0.7–0.9, with its converged mean around 0.98; the early-training trajectory shows a sharp initial drop followed by layers diverging from one another.
- **Diffusion** shows the most erratic behavior of any task: large swings between layers and, in at least one layer, a trajectory that climbs before reversing direction partway through training rather than settling smoothly. This is consistent with the elevated gradient-norm variance described above.
- **Language Modeling** shows a steady, monotonic decrease in α across its two layers, ending below 1.0 in both, with no oscillation at any point in training.
- **Corruption Robustness** shows layer depth mattering: earlier layers push α up while later layers pull it down.

We read these figures directly from our own logged trajectory data, so the qualitative pattern (task-dependent, non-random movement of α , with a consistent early/mid/late shape per task) is something we are confident in. We are less confident in treating every individual number above as precise to the decimal place reported by our plotting tool, and would recommend spot-checking a few of them against the raw trajectory logs before treating this as a headline finding, given that we already caught one plotting-description mismatch elsewhere in this project (Section VIII).

The original GoLU paper’s proposed mechanism is that output variance scales approximately with the square of the activation’s slope, $\text{Var}[f(x)] \approx f'(\mu)^2 \sigma^2$ [1]. Since α directly controls how steep the Gompertz gate is, our trajectory data suggests an interpretation consistent with that mechanism, though we have not measured it directly: Detection pushing α up to ~ 1.17 – 1.32 means a steeper gate than static GoLU, i.e. moving away from the paper’s variance-reduction effect (more GELU/Swish-like); Segmentation and Language Modeling pushing α down means *more* aggressive variance compression than static GoLU. We present this as a plausible reading consistent with the original paper’s own theory, not as something we have confirmed empirically – we did not log per-layer input/output variance for this project, so this remains a hypothesis. Directly testing it (logging activation variance

Gradient-Norm Stability: Static GoLU vs Alpha-GoLU

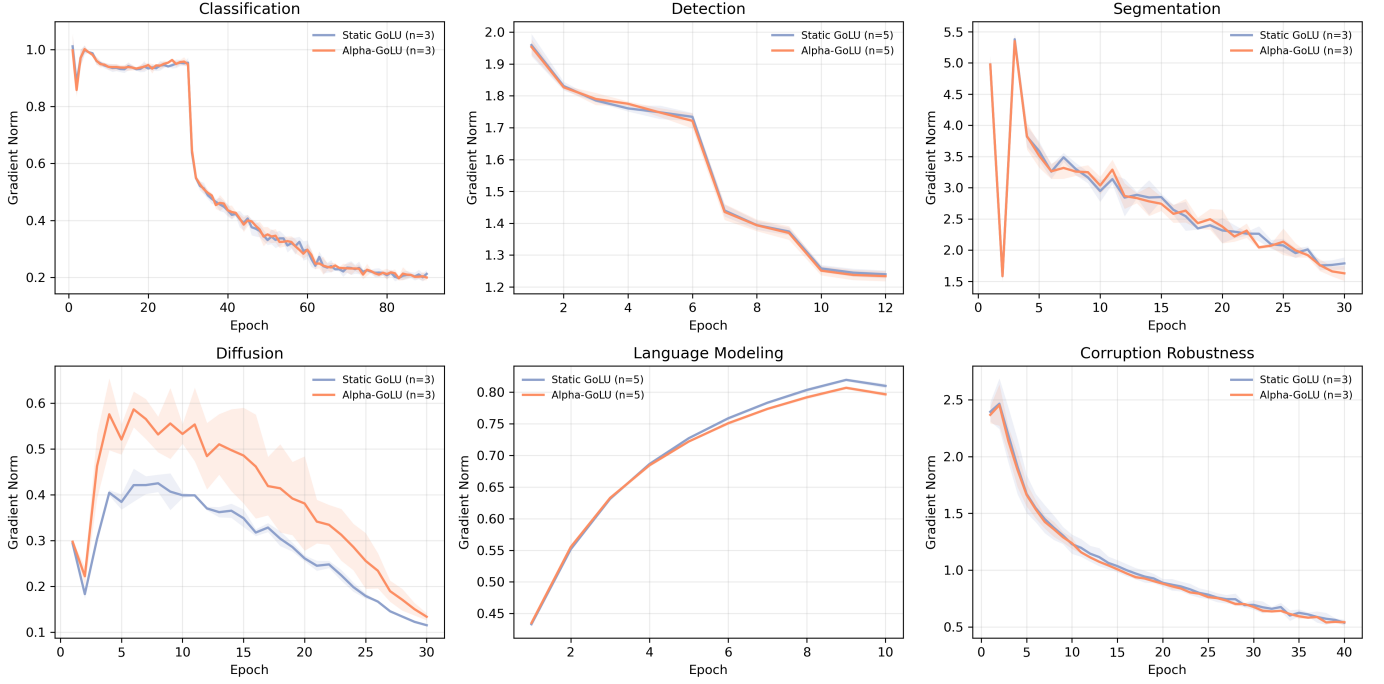


Fig. 3. Gradient norm vs. epoch, mean ± 1 std across seeds, static GoLU vs. Alpha-GoLU, one subplot per task.

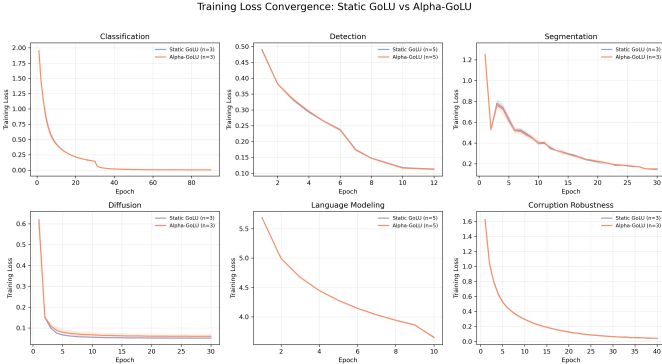


Fig. 4. Training loss vs. epoch, averaged over 3 seeds with a ± 1 standard deviation band, static GoLU vs. Alpha-GoLU.

per layer, the way the original paper’s Table 1 does) would be a natural, low-cost next step.

E. Corruption robustness figures (provisional)

Figures 7 and 8 break the robustness comparison down by corruption type and by retention ratio across all eight activations. As read off these figures, Alpha-GoLU and PReLU sit near the top of the retention-ratio ranking (each around 31%, close to ReLU), with Adaptive Swish lowest (around 27%); by corruption type, Alpha-GoLU looks strongest on blur and mid-table on Gaussian and shot noise, where PReLU leads. We are stating these as descriptive readings of the figures, not as a finding we are confident in. This is because

Section VI-A already established that our robustness evaluation is not currently reproducible run-to-run for a fixed seed and activation – the same command run twice gave different numbers for Alpha-GoLU. Since we do not know whether that non-determinism affects the per-corruption-type breakdown as well as the top-line accuracy, we are not willing to draw a conclusion (e.g. “Alpha-GoLU is more robust to blur”) from Figures 7–8 until the underlying reproducibility issue is fixed and the comparison is re-run cleanly.

VII. INTERPRETATION OF RESULTS

Taken together, the results in Table III are close to a null result on five of six tasks: on classification and diffusion, static GoLU is marginally (or, for diffusion, clearly) ahead; on segmentation, Alpha-GoLU is marginally ahead; on detection the two are indistinguishable. None of these five differences are large relative to their standard deviations, consistent with p -values well above 0.05. Language modeling is the one task where the difference, though numerically small (about 0.2% lower perplexity), is statistically significant ($p = 0.018$). We do not think it would be honest to describe Alpha-GoLU as a clear, general improvement over static GoLU based on this data – which is, in a sense, consistent with our earlier critical review [2], where we also found GoLU’s own reported gains over GELU/Swish to be small and often within the margin of error – but we also do not want to erase the one place where our own data does show a real, if modest, effect.

The original GoLU paper reports that GoLU needs a different, often higher, learning rate than an architecture’s default

Alpha-GoLU Curated Trajectories (Early / Mid / Late Layers)

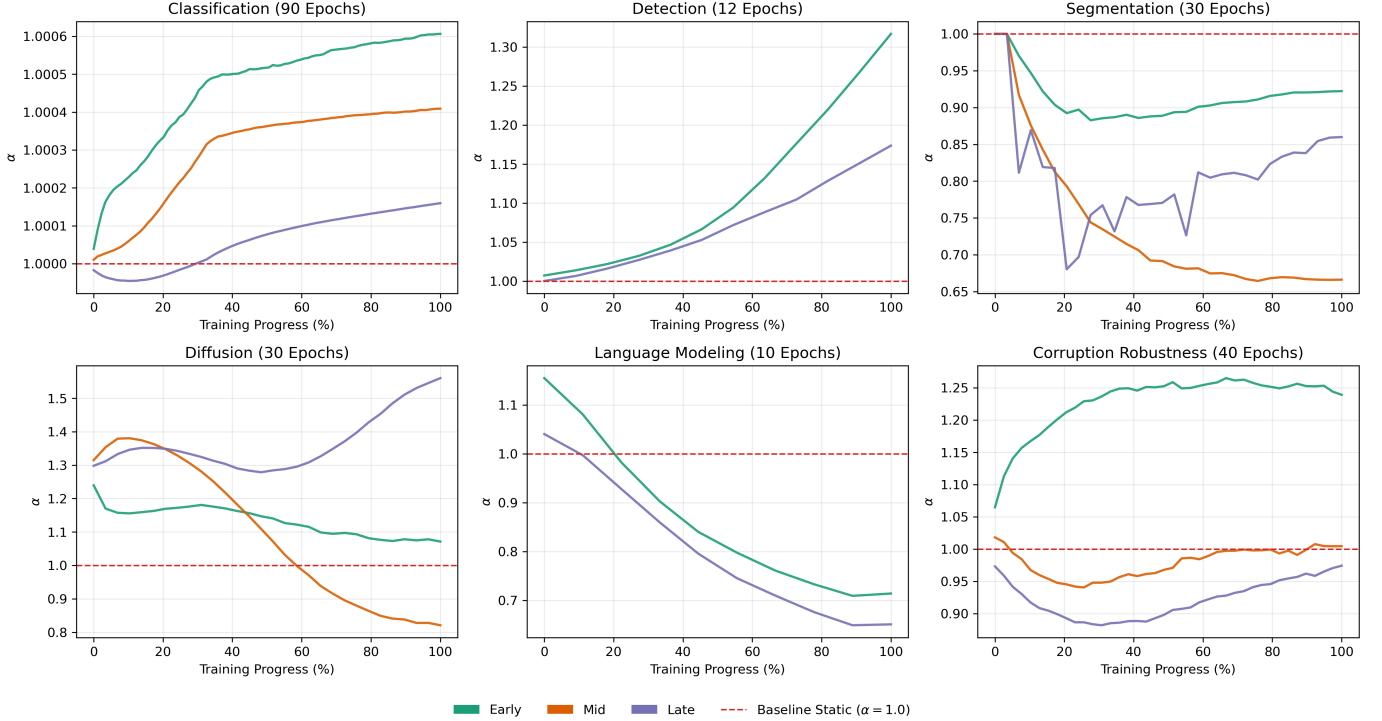


Fig. 5. Learned α value over training progress, curated to Early/Mid/Late representative layers per task. Detection and the language model only have 2 trackable layers, so these use fewer stages than the other tasks.

to show its best results – for example 0.01 vs. a default of 0.02 for semantic segmentation, and 0.001 vs. a default of 0.0003 for diffusion [1]. This is independent, external evidence that the whole GoLU family may be learning-rate-sensitive, which lines up with our own recurring learning-rate-related bugs (Section VIII: the segmentation α -LR configuration bug, and the language-model scale-up LR regression). It suggests that the hyperparameter sensitivity sweep already listed in our Limitations (Section IX) is probably the single highest-value follow-up experiment, rather than a generic gap.

The original paper itself also reports a negative result: in a Bayesian learning-curve-extrapolation case study (Appendix J), GoLU ranks sixth of seven activations at the optimal learning rate [1]. We read this as supporting evidence that our own mixed, mostly-null results are not a rebuttal of the original paper, but a consistent, more complete picture of an activation family whose benefits are real on some tasks but narrower than a first read of the main results might suggest.

One finding we do think is worth reporting on its own is the convergence behavior: even after averaging properly over three seeds, the training-loss curves for static GoLU and Alpha-GoLU overlap almost completely on most tasks. This means that if there is any benefit to the learned α , it is not showing up as faster or more stable optimization – it would have to show up in the final converged metric instead, and by that measure the effect is small and inconsistent across tasks.

The overhead results are more clear-cut: Alpha-GoLU is measurably slower than static GoLU and considerably slower than ReLU, mostly due to backward-pass cost. Combined with the lack of a consistent accuracy/robustness benefit, this is an important negative finding: on this set of tasks, the extra learnable parameter does not appear to be worth its extra compute cost.

We were not able to draw a firm conclusion on the corruption-robustness comparison. What started as a simple “we need one more number” gap turned out to be a more fundamental problem: repeating the same seed, activation, and pipeline twice gave two different scores (Section VI-A), meaning the evaluation itself is not currently reproducible. We do not report a robustness finding in this paper as a result, and treat it as an open item rather than a completed comparison.

VIII. CHALLENGES AND ENGINEERING PROCESS

A significant amount of the project’s effort went into finding and fixing correctness issues, which we think is worth documenting explicitly since it directly relates to the correctness of the implementation:

- An early bug in activation-parameter freezing caused several different activations to silently produce byte-identical results. This was caught before it could invalidate a full sweep of experiments.
- A configuration-resolution bug affected the learning rate used for α specifically in the segmentation task.

Converged Alpha Distribution by Task (alpha_golu)

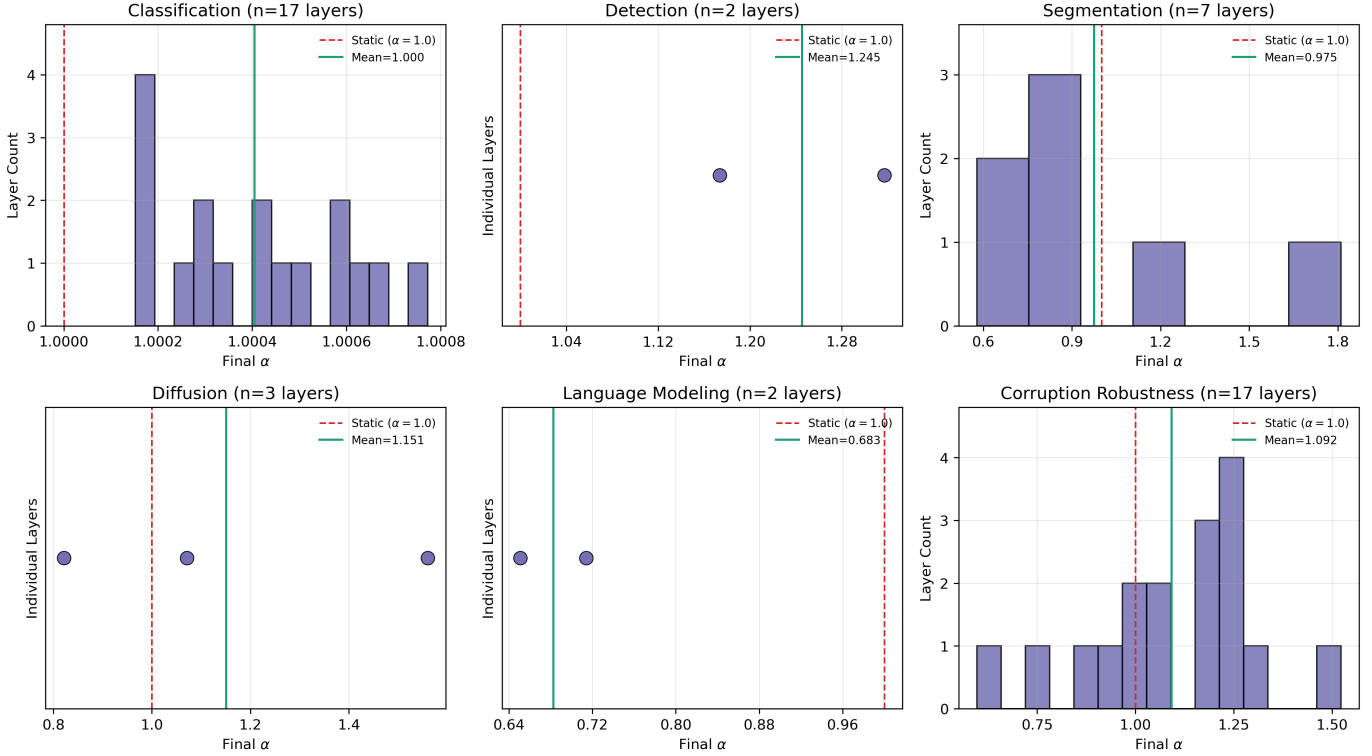


Fig. 6. Distribution of final, converged α values per task.

- A learning-rate tuning problem in the language-model scale-up task caused a real accuracy regression. This was root-caused, fixed (lower α learning rate of 2×10^{-5} , a 4-epoch warmup, and more conservative gradient clipping), and confirmed fixed with a paired significance test ($p = 0.55$, i.e. no longer distinguishable from noise).
- Several data-pipeline bugs made real, already-collected experimental data invisible or miscounted further downstream, including: a missing key in one runner's output that silently broke aggregation for that task; a legacy folder-naming split that scattered robustness results across two different folder names; a default results path that no runner ever actually wrote to, which made the summary tool falsely report zero results; a "most recent run" selection rule that could pick a stale, interrupted run instead of a completed one; and a stale import in a profiling script left over from an earlier refactor, which had silently broken that whole diagnostic tool for some time.
- A number of visualization bugs were fixed only after inspecting the actual rendered figures, not just the plotting code: unbounded figure sizes once more activations were added, mismatched y-axis scales that hid small-magnitude tasks, a matplotlib scientific-notation artifact, incorrect layer-stage labels for shallow architectures, and histogram artifacts caused by too few samples.
- The detection task's forward-slower-than-backward timing result looked suspicious at first, so instead of assuming it was a bug or assuming it was correct, we built a dedicated timing-verification tool using PyTorch's built-in profiler. That tool itself needed two rounds of debugging (a profiler attribute name that changed between PyTorch versions, and a missing GPU synchronization point between timed regions) before it could be trusted.
- While assembling the full results file for this report, we noticed that for the two tasks where seeds were collected across two separate runs (Detection and the Language Model, each of which has an original 3-seed set plus a later 2-seed addition), the automatically-computed summary mean/std in our results file used only the most recently added seeds rather than all five completed seeds on record. We traced this to its root cause: the statistics-building code filtered the complete per-seed record down to only the seeds passed to that specific command invocation (e.g. running `extra_seeds` with `--seeds 2024 2025` on a task that already had seeds 42/123/999 done), even though the complete per-seed record was always correctly stored on disk. We fixed the code to always rebuild statistics from every seed on record, confirmed the fix compiles and that the full unit test suite (25 tests) still passes, and used `python3 cli.py rebuild_summary` - which reads directly

Robustness Breakdown by Corruption Type

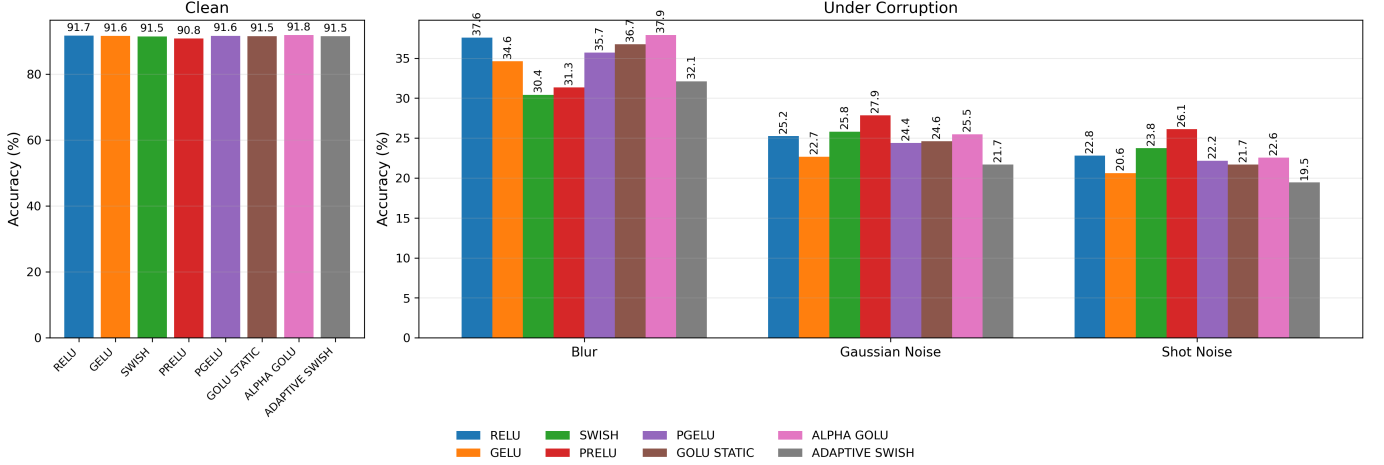


Fig. 7. Accuracy under each corruption type (clean, Gaussian noise, shot noise, blur), split into two panels because clean accuracy sits on a different scale from the corrupted conditions.

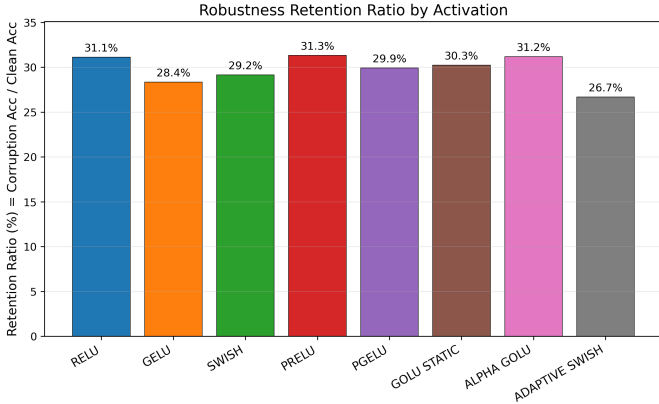


Fig. 8. Retention ratio (corrupted accuracy divided by clean accuracy) per activation, separating “degrades less under noise” from “starts higher.”

from the saved per-run result files and was unaffected by this particular bug – to regenerate the statistics used throughout this report. The regenerated values matched our manually recomputed 5-seed statistics for Detection and the Language Model exactly, which we take as independent confirmation that both the fix and our manual check were correct.

We include this list because we think that, given the results in Section VI are mostly a null result, the value of this project lies at least as much in building a correct, reproducible experimental pipeline as in the specific numbers it produced.

IX. LIMITATIONS AND FUTURE WORK

The following gaps are known and explicitly not covered by this report:

- **Training time.** We do not have trustworthy total training-time numbers per task; the only wall-clock timing we

observed directly came from an early, likely CPU-only local test run that is not representative of the real GPU training runs. Real per-task timing is stored in each run’s results file and should be pulled from there rather than estimated.

- **Incomplete and non-reproducible robustness coverage.** The corruption-robustness comparison is the most significant open item, and it turned out to be a deeper problem than a simple missing re-run: repeating the same seed, activation, and pipeline twice produced two different scores (Section VI-A), meaning the evaluation itself is not currently reproducible. This needs to be root-caused (most likely candidates: unseeded randomness in corruption generation, or the eval depending on whichever trained-model checkpoint happens to be loaded) before any static-vs-Alpha robustness claim can be made with confidence.
- **Overhead verification not yet extended to static/Alpha-GoLU.** The forward- and backward-pass discrepancy between our overhead tracker and the dedicated `torch.profiler` verification tool is now resolved for Detection (Section VI-B), confirmed with a fresh CUDA-event-timed run. That confirming run used ReLU as the test case; we have not yet repeated it for static GoLU and Alpha-GoLU specifically, so we do not have an exact updated profiler number for those two activations, only the shared code-path fix and the ReLU evidence that it works.
- **Phase 2 significance testing.** The Phase 2 scale-up results (Table II), including the language-model regression noted there, have not yet been checked with a significance test.
- **Qualitative outputs.** We did not generate qualitative examples (segmentation masks, detection boxes, sample diffusion images); this would require changes to the

runner code and additional runs.

- **Hyperparameter sensitivity.** We did not run a sensitivity sweep over the α initialization value or α -specific learning rate; this would require a new experiment campaign.
- **Effect size figure.** Cohen’s d is computed by our statistics module but has not yet been turned into a figure.
- **No reproduction of the original GoLU experiments, scale gap quantified.** Consistent with our earlier critical review [2], this project builds on the original authors’ reported numbers rather than reproducing their ImageNet/COCO-scale experiments directly, due to compute constraints. To state that gap concretely rather than only in general terms: the original paper’s experiments used approximately 112,000 NVIDIA A100 GPU-hours across nine or more ImageNet-1k classification architectures plus MS-COCO-scale detection, segmentation, and diffusion [1]. Our own experiments run on a single NVIDIA L40S (46 GB), with a rough wall-clock estimate, summed across all saved run directories via a simple timestamp-based script, on the order of several thousand GPU-hours. That script’s own output explicitly flags itself as approximate rather than a rigorous profiler measurement, and inspecting the per-run breakdown confirms this: a handful of entries show implausible values (700+ hours for a single seed) that are clearly artifacts of stale or archived run directories rather than real continuous compute, most likely inflated by an early period, before we implemented epoch- and seed-level resume, when server disconnects meant some long runs (e.g. Detection, which took close to 10 hours per seed) had to be restarted from scratch. We report this as an order-of-magnitude comparison rather than a precise figure: even generously interpreted, our total compute is several orders of magnitude smaller than the original paper’s 112,000 A100 GPU-hours, across a correspondingly much smaller set of datasets and architectures.

X. CONCLUSION

We implemented Alpha-GoLU, a per-layer learnable generalization of the GoLU activation function, motivated directly by a gap identified in our earlier critical review of the original paper [2], and evaluated it against seven other activations across six tasks with a full statistical testing and profiling pipeline. The core hypothesis – that a learnable gate steepness would give a consistent benefit over a fixed one – is not supported on five of the six tasks: differences between static GoLU and Alpha-GoLU are small, inconsistent in direction, and not statistically significant, training curves and gradient norms are nearly identical, and Alpha-GoLU adds a real, measurable computational cost. Language modeling is the exception, with a small but statistically significant improvement, and diffusion is a genuine regression accompanied by higher, more variable gradient norms during training – both worth following up on rather than averaging away. We report this mixed picture rather than reframing it as a clean success or a clean failure, and we believe the more solid contribution

of this project is the experimental infrastructure itself: a reproducible, publicly available pipeline with checkpointing, proper multi-seed statistical testing, overhead profiling, and a documented process of finding and fixing real correctness bugs along the way – including a subtle statistics-aggregation bug that was silently understating some of our own sample sizes. Future work would need to close the gaps listed in Section IX, in particular a reproducible robustness evaluation and a hyperparameter sensitivity study, before any stronger claim about Alpha-GoLU could be made.

REFERENCES

- [1] I. Das, M. Safari, S. Adriaensen, and F. Hutter, “Gompertz linear units: Leveraging asymmetry for enhanced learning dynamics,” arXiv preprint arXiv:2502.03654v2, May 2025.
- [2] D. Kopic, “Critical Analysis of Gompertz Linear Units (GoLU) for Enhanced Learning Dynamics,” Seminar in AI course report, University of Sarajevo, 2026.
- [3] V. Nair and G. E. Hinton, “Rectified Linear Units Improve Restricted Boltzmann Machines,” in *Proc. ICML*, 2010, pp. 807–814.
- [4] D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs),” arXiv preprint arXiv:1606.08415, 2016.
- [5] P. Ramachandran, B. Zoph, and Q. V. Le, “Searching for Activation Functions,” arXiv preprint arXiv:1710.05941, 2017.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification,” in *Proc. ICCV*, 2015, pp. 1026–1034.
- [7] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” Tech. Report, University of Toronto, 2009.
- [8] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, vol. 88, pp. 303–338, 2010.